

Propuesta de Proyecto
Especificación de Requisitos de Software
Sistema de Gestión Bibliotecaria - UNILIB

Ingeniería de Software - RQY1102

Actualización según Rubrica 3: Prototipo, Plan de Pruebas y Control de Versiones

Planificación, Especificación de Requisitos, Modelado UML, Vistas Arquitectónicas,
Prototipado, Pruebas y Aseguramiento de Calidad

DUOC UC - 2026

Indice de Contenidos

Contenido

Contenido.....	2
Ficha del Documento	4
Integrantes del equipo	4
1. Introducción.....	5
1.1. Propósito	5
1.2. Ámbito del Sistema	5
1.3. Definiciones, Acronimos y Abreviaturas	6
1.4. Referencias.....	6
1.5. Vision General del Documento.....	7
2. Descripción General	8
2.1. Perspectiva del Producto	8
2.2. Funciones del Producto	8
2.3. Características de los Usuarios.....	8
2.4. Restricciones	9
2.5. Suposiciones y Dependencias	9
2.6. Requisitos Futuros	9
3. Requisitos Específicos	10
3.1. Requisitos Comunes de las Interfaces	10
3.1.1. Interfaces de Usuario	10
3.2. Requisitos Funcionales	11
3.3. Requisitos No Funcionales	12
3.4. Otros Requisitos	12
4. Modelado de Software, Principios de Diseño y Arquitectura.....	12
4.1. Técnicas de Modelado UML Aplicadas	13
4.2. Principios Fundamentales de Diseño Aplicados.....	13
4.3. Diagrama de Casos de Uso	14
4.4. Diagrama de Clases	15
4.5. Diagrama de Componentes	16
4.6. Diagrama de Actividad	17
4.7. Diagrama de Secuencia – Solicitud de Préstamo.....	18
4.8. Diagrama de Actividad – Registro de Devolución.....	19
4.9. Diagrama de Secuencia – Registro de Devolución	20

4.10. Diagrama de Despliegue.....	21
4.11. Modelo Arquitectónico 4+1.....	21
4.12. Patrones de Diseño y Arquitectónicos Aplicados	22
5. Propuesta de Planificación	27
5.1. Descripción General de la Planificación	28
5.2. Definición del Equipo de Trabajo	28
5.3. Definición de Actividades Principales del Proyecto	28
5.4. Tablero Kanban - Work Ítems del Proyecto.....	29
5.5. Carta Gantt	29
5.6. CI/CD y Buenas Prácticas de Desarrollo Seguro	29
6. Estándar de Calidad Aplicado - ISO/IEC 25010.....	31
6.1. Selección del Estándar entre ISO, Nielsen y WCAG	31
6.2. Aplicación de ISO/IEC 25010 en UNILIB	31
6.3. Complemento de Usabilidad: Heurísticas de Nielsen	32
6.4. Complemento de Accesibilidad: WCAG 2.1 Nivel AA.....	32
6.5. Matriz de Verificación de Calidad.....	32
7. Anexos	33
7.1. Matriz de Especificación de Requisitos	33
7.2. Matriz de Cumplimiento de Rubrica 2	35
8. Propuestas de Mejora y Aseguramiento de Calidad	37
8.1. Mejoras Funcionales.....	37
8.2. Mejoras Técnicas.....	37
8.3. Aseguramiento de Calidad	37
9. Conclusión	38

Ficha del Documento

Fecha	Revision	Autor	Modificacion
			Actualización del informe con prototipado funcional, plan de pruebas y control de versiones.

Documento validado por las partes en fecha: _____

Integrantes del equipo

Nombre Integrante del Equipo	Rol Definido
Ignacio Arntz	Arquitecto de Software / Líder Técnico
José Díaz	Desarrollador Backend / API REST
Simón González	Desarrollador Frontend / UX/UI
Andrés Mendoza	QA / Control de Calidad

1. Introducción

El presente documento constituye la especificación de Requisitos de Software (ERS) del proyecto UNILIB, Sistema de Gestión Bibliotecaria, desarrollado en el contexto del curso Ingeniería de Software (RQY1102) de DUOC UC. El documento ha sido elaborado siguiendo el estándar IEEE 830 e incorpora principios de calidad ISO 9000, gestión de proyectos PMI, modelado UML, arquitectura 4+1 y criterios de calidad ISO/IEC 25010.

La ERS describe de manera formal y detallada los requisitos funcionales y no funcionales que debiera satisfacer el sistema, junto con el contexto operacional, los actores involucrados, el diseño arquitectónico, los diagramas UML utilizados y la planificación del proyecto. La presente versión incorpora además un prototipo funcional, un plan de pruebas y un esquema de control de versiones para dar cumplimiento a la rúbrica 3.

1.1. Propósito

El propósito de este documento es definir con precisión los requisitos del sistema UNILIB, proporcionando una base formal y verificable para el diseño, desarrollo, pruebas e implementación del software. Este documento está dirigido a:

- Equipo de desarrollo: para comprender el alcance funcional y técnico del sistema.
- Clientes y usuarios finales: para validar que el sistema responde a sus necesidades reales.
- Equipo de QA: para diseñar los planes de prueba que verifiquen el cumplimiento de los requisitos.
- Docentes evaluadores: para revisar el nivel de análisis, especificación, modelado y arquitectura alcanzada.

1.2. Ámbito del Sistema

El sistema recibirá el nombre de UNILIB - Sistema de Gestión Bibliotecaria.

Lo que el sistema hará:

- Gestionar el catálogo de libros y materiales bibliográficos de la biblioteca.
- Administrar usuarios: estudiantes, docentes, bibliotecarios y administradores.
- Controlar el ciclo completo de préstamos: solicitud, aprobación, devolución, renovación y multas.
- Enviar notificaciones automáticas ante vencimientos y disponibilidad de reservas.
- Generar reportes estadísticos exportables para la toma de decisiones.

Lo que el sistema no hará:

- No gestionara procesos de adquisición o compra de nuevos materiales bibliográficos.
- No integrara módulos de pago en línea para multas en esta versión.
- No administrara recursos físicos distintos al acervo bibliográfico registrado.

Beneficios esperados:

- Reducción del tiempo operativo en préstamos y devoluciones en al menos un 60%.
- Eliminación de registros duplicados o inconsistentes en el inventario.
- Mejora en la satisfacción del usuario mediante acceso en línea y notificaciones proactivas.

- Generación de estadísticas que apoyen la gestión y adquisición de materiales.

1.3. Definiciones, Acrónimos y Abreviaturas

Termino	Definición
ERS	Especificación de Requisitos de Software
RF	Requisito Funcional
RNF	Requisito No Funcional
IEEE 830	Estándar para especificaciones de requisitos de software
UNILIB	Nombre del sistema: Sistema de Gestión Bibliotecaria universitaria
JWT	JSON Web Token - mecanismo de autenticación stateless
TLS	Transport Layer Security - protocolo de cifrado de comunicaciones
API REST	Interfaz de programación de aplicaciones basada en arquitectura REST
UX/UI	User Experience / User Interface
QA	Quality Assurance - Aseguramiento de Calidad
PMI	Project Management Institute
CRUD	Create, Read, Update, Delete - operaciones básicas de persistencia
UML	Lenguaje Unificado de Modelado para especificar, visualizar y documentar sistemas
4+1	Modelo de vistas arquitectónicas: lógica, procesos, desarrollo, física y escenarios
ISO/IEC 25010	Modelo de calidad de producto software usado como estándar principal en UNILIB
WCAG	Pautas de accesibilidad web basadas en principios perceptible, operable, comprensible y robusto
Nielsen	Heurísticas de usabilidad usadas como apoyo para evaluar la experiencia de usuario
CI/CD	Integración y despliegue continuo para automatizar pruebas y entregas

1.4. Referencias

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC 9001:2015: Sistemas de gestión de la calidad - Requisitos.
- PMI - A Guide to the Project Management Body of Knowledge (PMBOK Guide), 6.a edición.
- ISO/IEC 25010: Systems and software engineering - Systems and software Quality Requirements and Evaluation (Square).
- Lenguaje Unificado de Modelado (UML) aplicado a diagramas estructurales y de comportamiento.
- Modelo de vistas arquitectónicas 4+1 propuesto por Philippe Kruchten.
- Heurísticas de usabilidad de Jakob Nielsen.
- WCAG 2.1 nivel AA como criterio complementario de accesibilidad web.
- Material del curso Ingeniería de Software RQY1102, DUOC UC, 2026.

1.5. Vision General del Documento

1. Introducción: propósito, ámbito, definiciones, referencias y organización del documento.
2. Descripción General: perspectiva del producto, funciones, usuarios, restricciones, suposiciones y requisitos futuros.
3. Requisitos Específicos: interfaces, requisitos funcionales, no funcionales y otros requisitos.
4. Modelado de Software, Principios de diseño y Arquitectura: UML, principios de diseño, patrones y modelo 4+1.
5. Propuesta de planificación: metodología, equipo, actividades, Kanban, Carta Gantt y practicas CI/CD.
6. Estándar de Calidad Aplicado: selección y aplicación de ISO/IEC 25010 con apoyo de Nielsen y WCAG.
7. Anexos: matriz de requisitos y matriz de cumplimiento de rubrica.
8. Complementos de la Rúbrica 3: prototipo, plan de pruebas y control de versiones.
9. Propuestas de Mejora y Aseguramiento de Calidad.
10. Conclusión.

2. Descripción General

Esta sección describe los factores de contexto que afectan al producto y a sus requisitos. No constituye una especificación de requisitos, sino el marco que permite comprenderlos y contextualizarlos adecuadamente.

2.1. Perspectiva del Producto

UNILIB es un sistema web independiente, diseñado para operar de forma autónoma dentro de la infraestructura tecnológica de la institución educativa. No depende de otros sistemas externos para su funcionamiento esencial, aunque esta diseñado para integrarse con:

- Servidor de correo electrónico institucional: para el envío de notificaciones automáticas.
- Sistemas de identidad institucional (futuro): posibilidad de integración con SSO (Single Sign-On) en versiones posteriores.

El sistema opera bajo una arquitectura cliente-servidor. El cliente accede al sistema mediante un navegador web sin necesidad de instalación. El servidor centraliza la lógica de negocio y la base de datos relacional. La comunicación entre capas se realiza mediante una API REST con autenticación JWT.

2.2. Funciones del Producto

- Gestión de usuarios: registro, autenticación, modificación de perfiles y control de acceso basado en roles.
- Gestión del catálogo bibliográfico: alta, edición y baja de materiales con trazabilidad de ejemplares.
- Ciclo de préstamos: solicitud, aprobación, renovación, devolución y cálculo de multas.
- Sistema de reservas: cola de espera con notificación automática de disponibilidad.
- Notificaciones: alertas proactivas por correo ante eventos relevantes del sistema.
- Reportes estadísticos: Dashboard y exportaciones para la gestión de la colección y el personal.

2.3. Características de los Usuarios

Usuario	Perfil / Tipo	Descripción
Estudiante	Usuario final básico	Accede al catalogo, solicita prestamos, consulta su historial y recibe notificaciones de vencimiento.
Docente	Usuario final avanzado	Solicita reservas, accede a colecciones especializadas y gestiona prestamos de mayor duración.
Bibliotecario	Operador del sistema	Gestiona prestamos, devoluciones, actualiza el catálogo y genera reportes operativos.
Administrador	Superusuario	Administra usuarios, roles, parámetros del sistema y supervisa auditoria y seguridad.

2.4. Restricciones

- El sistema debera desarrollarse utilizando tecnologías web estándar (HTML5, CSS3, JavaScript) con un framework backend a definir por el equipo.
- La base de datos debera ser relacional (MySQL o PostgreSQL) para garantizar integridad referencial.
- El sistema debera desplegarse en un servidor con acceso a internet para permitir el acceso remoto.
- Todas las comunicaciones deberan realizarse bajo protocolo HTTPS (TLS 1.2 o superior).
- El sistema debera cumplir con la normativa institucional de protección de datos personales.
- El equipo de desarrollo trabajara con dedicación parcial durante 8 semanas.
- No se cuenta con presupuesto para licencias de software propietario; se priorizarán herramientas de código abierto.

2.5. Suposiciones y Dependencias

- La institución cuenta con infraestructura de servidor capaz de alojar una aplicación web y una base de datos relacional.
- Los usuarios finales disponen de dispositivos con acceso a internet y un navegador web actualizado.
- El servidor de correo institucional estará disponible para el envío de notificaciones automáticas.
- Los datos iniciales del catálogo bibliográfico serán proporcionados por la biblioteca en formato estructurado (Excel/CSV) para su migración al sistema.
- Los requisitos definidos en este documento son estables. Cambios significativos en el alcance requerirán una revisión formal del ERS.

2.6. Requisitos Futuros

- Módulo de pagos en línea para multas y sanciones mediante pasarela de pago.
- Integración con el sistema de identidad institucional para inicio de sesión único (SSO/OAuth2).
- Aplicación móvil nativa (iOS y Android) para acceso al catálogo y gestión de préstamos.
- Módulo de adquisiciones: gestión del proceso de compra y recepción de nuevos materiales.
- Sistema de recomendaciones bibliográficas basado en historial de préstamos del usuario.
- API publica para integración con repositorios bibliográficas externos (MARC21, Dublin Core).

3. Requisitos Específicos

Esta sección contiene los requisitos a un nivel de detalle suficiente para permitir al equipo de diseño y desarrollo construir un sistema que los satisfaga, y al equipo de QA planificar las pruebas que verifiquen su cumplimiento. Todos los requisitos son unívocamente identificables mediante un Código único (RF-XX para funcionales, RNF-XX para no funcionales).

3.1. Requisitos Comunes de las Interfaces

3.1.1. Interfaces de Usuario

- Diseño responsive: la interfaz se adaptará automáticamente a resoluciones de pantalla de escritorio ($\geq 1280\text{px}$), tablet ($768\text{px} - 1279\text{px}$) y móvil ($< 768\text{px}$).
- Paleta de colores institucional: azul primario (#1C5E82), blanco (FFFFFF) y grises neutros para fondos y texto secundario.
- Tipografía legible: uso de fuente sans-serif (Arial o equivalente), tamaño mínimo de 14px para texto de cuerpo en interfaz.
- Navegación consistente: menú de navegación principal visible en todas las páginas con indicación del módulo activo.
- Mensajes de retroalimentación: toda acción del usuario debera producir una respuesta visual en un tiempo máximo de 2 segundos.
- Accesibilidad básica: contraste de color mínimo de 4.5:1 según WCAG 2.1 nivel AA.

3.2. Requisitos Funcionales

ID	Nombre	Descripcion	Prioridad	Estado
RF-01	Registro de usuarios	El sistema permitirá crear, modificar y dar de baja cuentas de usuario con validación de datos obligatorios y asignación de rol.	Alta	Propuesto
RF-02	Autenticación	El sistema proveerá inicio de sesión seguro con credenciales únicas. Las contraseñas deberán almacenarse cifradas.	Alta	Propuesto
RF-03	Busqueda en catalogo	El sistema permitirá buscar materiales por título, autor, ISBN, editorial y categoría, con filtro de disponibilidad en tiempo real.	Alta	Propuesto
RF-04	Solicitud de prestamos	El sistema permitirá a los usuarios solicitar préstamos en línea. El bibliotecario aprobará o rechazará la solicitud.	Alta	Propuesto
RF-05	Registro de devoluciones	El sistema registrará la entrega de materiales, verificará fecha de vencimiento y calculará multas automáticamente si corresponde.	Alta	Propuesto
RF-06	renovación de prestamos	El usuario podrá renovar un préstamo activo en línea, siempre que no existan reservas pendientes para el material.	Media	Propuesto
RF-07	Reservas de materiales	El sistema permitirá reservar materiales no disponibles. Al retornar el material, se notificará automáticamente al siguiente en lista.	Media	Propuesto
RF-08	Notificaciones automáticas	El sistema enviará alertas por correo electrónico ante vencimientos próximos, disponibilidad de reservas y multas pendientes.	Alta	Propuesto
RF-09	Gestión del catalogo	El bibliotecario podrá agregar, editar y eliminar registros bibliográficos incluyendo datos de ejemplares y estado de disponibilidad.	Alta	Propuesto
RF-10	Generación de reportes	El sistema generará reportes estadísticos exportables en PDF/Excel: materiales más prestados, usuarios activos y tasa de devoluciones.	Media	Propuesto
RF-11	Gestión de multas	El sistema permitirá calcular, registrar y consultar multas generadas por retrasos en la devolución de materiales.	Alta	Propuesto
RF-12	Historial de usuario	El sistema permitirá visualizar el historial completo de préstamos, devoluciones y multas de cada usuario.	Media	Propuesto
RF-13	Auditoria del sistema	El sistema registrará todas las acciones críticas realizadas por los usuarios (login, modificaciones, eliminaciones).	Alta	Propuesto
RF-14	Validación de datos	El sistema validará todos los datos ingresados por el usuario para evitar inconsistencias (campos obligatorios, formatos, duplicados).	Alta	Propuesto

3.3. Requisitos No Funcionales

ID	Nombre	Descripcion	Categoria	Estado
RNF-01	Rendimiento	El tiempo de respuesta para operaciones estándar no debera superar 2 segundos bajo carga normal.	Eficiencia	Propuesto
RNF-02	Disponibilidad	El sistema debera operar con uptime mínimo del 99.5%. El tiempo de recuperación ante fallos no superara las 2 horas.	Confiabilidad	Propuesto
RNF-03	Seguridad	El sistema implementara autenticación JWT, cifrado TLS en tránsito, cifrado de contraseñas con bcrypt y control de acceso por roles.	Seguridad	Propuesto
RNF-04	Usabilidad	La interfaz será intuitiva, con curva de aprendizaje inferior a 1 hora para usuarios básicos, siguiendo principios de UX/UI.	Usabilidad	Propuesto
RNF-05	Accesibilidad	El sistema será responsive y compatible con navegadores modernos (Chrome, Firefox, Edge, Safari) y dispositivos móviles.	Portabilidad	Propuesto
RNF-06	Mantenibilidad	El código debera estar documentado con cobertura de pruebas automatizadas $\geq 80\%$ y arquitectura modular.	Mantenibilidad	Propuesto
RNF-07	Escalabilidad	La arquitectura debera soportar un crecimiento del 50% en el número de usuarios sin degradación del rendimiento.	Eficiencia	Propuesto

3.4. Otros Requisitos

- Respaldo de datos: política de respaldo automático de la base de datos con frecuencia mínima diaria y retención de al menos 30 días.
- Auditoria: registro en log de operaciones críticas con marca de tiempo y usuario responsable.
- Internacionalización: el sistema operara en idioma español y se diseñara para incorporar otros idiomas en el futuro sin refactorización mayor.

4. Modelado de Software, Principios de Diseño y Arquitectura

Esta sección incorpora los elementos centrales de la Unidad 2 para adecuar el informe a la rúbrica: técnicas de modelado UML, principios fundamentales de diseño, patrones arquitectónicos y modelo de vistas 4+1. Su objetivo es representar la arquitectura y el diseño de UNILIB de manera estandarizada, coherente y verificable.

4.1. Técnicas de Modelado UML Aplicadas

UML se utiliza como notación estándar para especificar, visualizar y documentar el sistema. En UNILIB se aplican diagramas estructurales y de comportamiento para cubrir tanto la organización interna del software como los flujos que ejecutan los usuarios.

Diagrama UML	Tipo	Objetivo en UNILIB
Caso de Uso	Comportamiento	Representa actores, funcionalidades principales y límites del sistema.
Clases	Estructural	Define entidades, atributos, operaciones y relaciones del dominio bibliotecario.
Componentes	Estructural	Muestra la organización lógica del software y sus dependencias.
Actividad	Comportamiento	Modela el flujo de trabajo de procesos críticos como la solicitud de préstamos y el registro de devoluciones.
Secuencia	Comportamiento	Representa la interacción temporal entre actores y componentes durante los procesos principales del sistema.
Despliegue	Estructural	Representa la distribución física del sistema en nodos de infraestructura.

4.2. Principios Fundamentales de Diseño Aplicados

Principio	Aplicacion en UNILIB	Contribucion a la calidad
Abstracción	Se representan entidades generales como Usuario, Libro, Prestamo y Reporte sin exponer detalles innecesarios de implementación.	Reduce complejidad y facilita la comunicación entre equipo técnico y stakeholders.
Encapsulamiento	Cada módulo oculta su lógica interna mediante servicios y API; por ejemplo, el cálculo de multas queda dentro del módulo de préstamos/multas.	Disminuye errores y evita dependencias innecesarias.
Modularidad	El sistema se divide en módulos: Autenticación, usuarios, catalogo, prestamos, reservas, notificaciones y reportes.	Facilita mantenimiento, pruebas y evolución incremental.
Alta cohesión	Cada componente agrupa responsabilidades relacionadas, como Catalogo para libros y ejemplares.	Mejora claridad del diseño y reduce dispersión funcional.
Bajo acoplamiento	Los módulos se comunican mediante API REST y contratos definidos, evitando acceso directo a lógica interna.	Permite cambiar un componente sin afectar todo el sistema.

4.3. Diagrama de Casos de Uso

El diagrama de casos de uso identifica los actores principales del sistema y las funcionalidades que cada uno puede ejecutar. Permite validar que los requisitos funcionales cubren las necesidades de estudiantes, docentes, bibliotecarios y administradores.

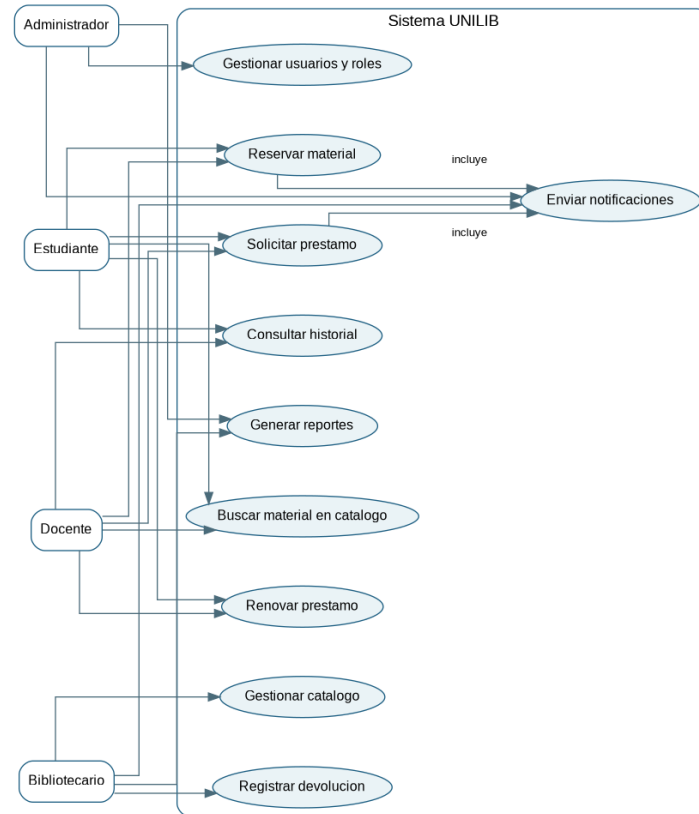


Figura 1. Diagrama de Casos de Uso de UNILIB.

El estudiante y el docente interactúan con el catálogo, solicitudes, renovaciones, reservas e historial. El bibliotecario gestiona el catálogo, devoluciones y reportes operativos. El administrador gestiona usuarios, roles, reportes y parámetros de control. Las notificaciones se consideran un caso incluido porque son disparadas por préstamos, reservas y vencimientos.

4.4. Diagrama de Clases

El diagrama de clases representa la estructura estática del dominio. Muestra las entidades principales, sus atributos, operaciones y relaciones. Esta vista

es la base para el modelo de datos relacional y la lógica de negocio.

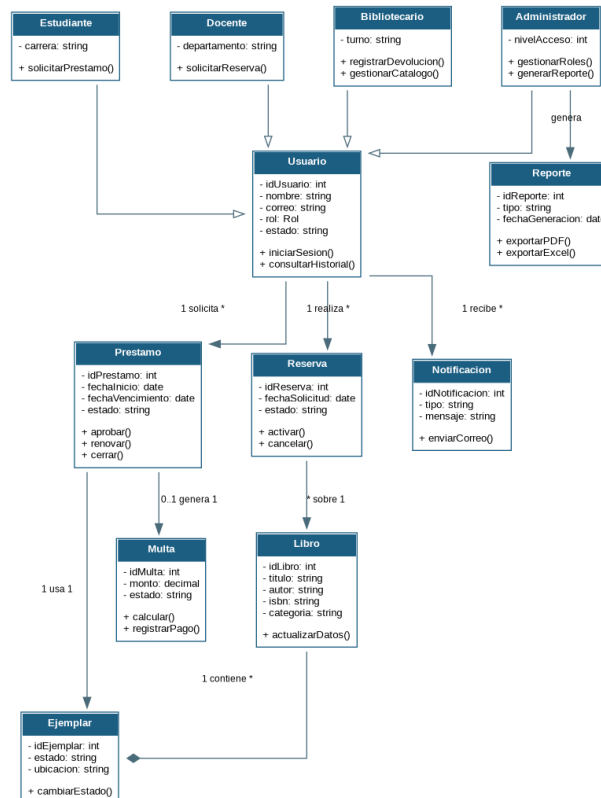


Figura 2. Diagrama de Clases de UNILIB.

La clase Usuario se especializa en Estudiante, Docente, Bibliotecario y Administrador. Libro contiene uno o más Ejemplares. Un Usuario puede tener varios Prestamos y Reservas. Un Prestamo puede generar una Multa y el sistema puede enviar Notificaciones asociadas a eventos del usuario.

4.5. Diagrama de Componentes

El diagrama de componentes describe la organización lógica del software. Se propone una arquitectura modular donde el Frontend se comunica con una API REST, y la API coordina servicios especializados.

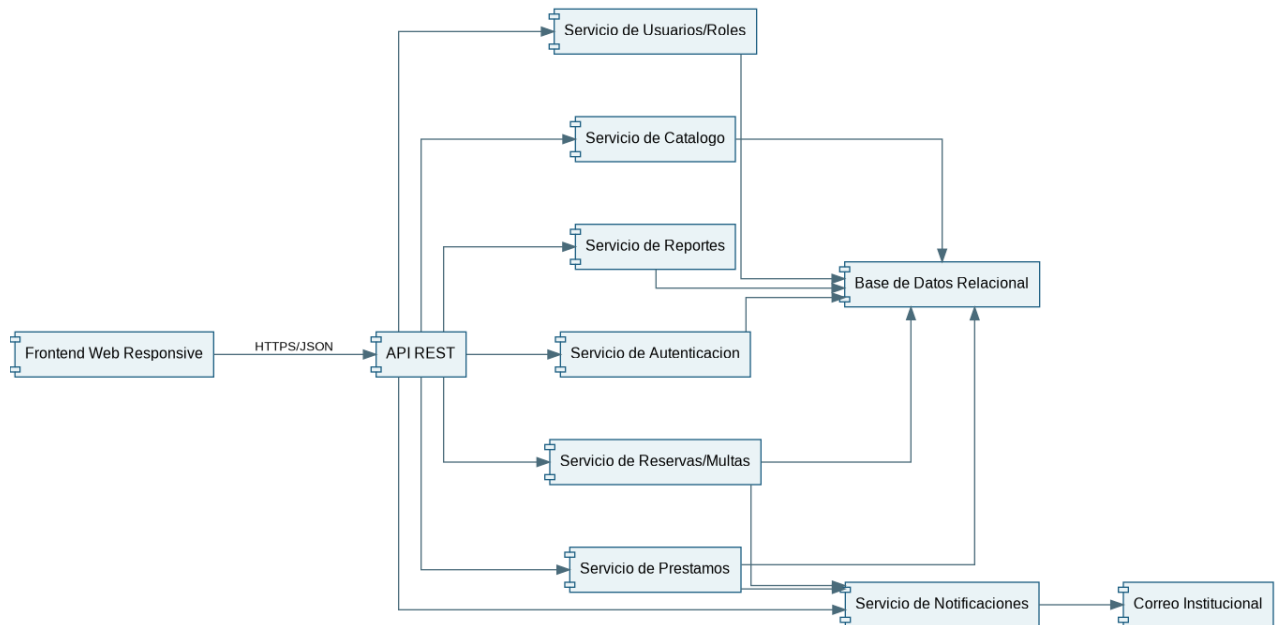


Figura 3. Diagrama de Componentes de UNILIB.

Esta división promueve modularidad, alta cohesión y bajo acoplamiento. Cada servicio se encarga de una responsabilidad específica y comparte información mediante contratos definidos. La base de datos relacional centraliza la persistencia y el servidor de correo permite notificaciones automáticas.

4.6. Diagrama de Actividad

El diagrama de actividad modela el flujo de trabajo de una solicitud de préstamo. Este flujo permite verificar condiciones, decisiones y respuestas del sistema antes de implementar la funcionalidad.

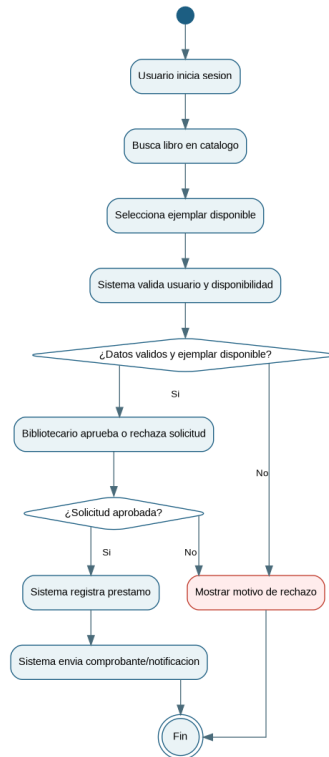


Figura 4. Diagrama de Actividad - Solicitud de Prestamo.

El proceso comienza con la autenticación del usuario, continua con la búsqueda y selección del material, valida disponibilidad y reglas de usuario, y termina con aprobación, registro del préstamo y envío de notificación. En caso de rechazo, el sistema entrega retroalimentación clara al usuario.

4.7. Diagrama de Secuencia – Solicitud de Préstamo

El diagrama de secuencia representa la interacción temporal entre los diferentes componentes del sistema durante el proceso de solicitud de préstamo de material bibliográfico. En él se observa cómo el estudiante realiza la solicitud mediante la interfaz de usuario, la cual se comunica con la API REST para validar la disponibilidad del ejemplar y registrar la operación correspondiente.

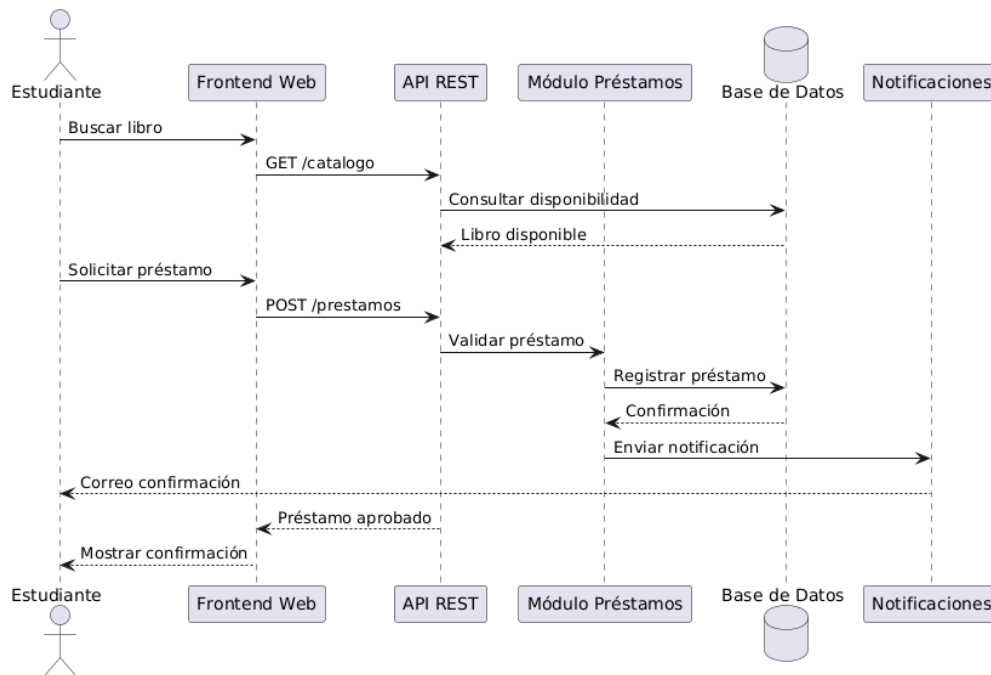


Figura 5. Diagrama de Secuencia - Solicitud de Préstamo.

Este diagrama permite visualizar el intercambio de mensajes entre los componentes de la arquitectura, facilitando la comprensión de las responsabilidades de cada módulo y asegurando la correcta ejecución de las reglas de negocio asociadas al préstamo de material bibliográfico.

Durante el proceso participan los servicios encargados de la gestión de préstamos, la base de datos y el módulo de notificaciones, permitiendo verificar la disponibilidad del recurso, almacenar la información del préstamo y comunicar la confirmación al usuario.

4.8. Diagrama de Actividad – Registro de Devolución

El diagrama de actividad representa el flujo de trabajo asociado al proceso de devolución de material bibliográfico dentro del sistema UNILIB. El proceso inicia cuando el bibliotecario recibe el ejemplar devuelto y localiza el préstamo correspondiente. Posteriormente, el sistema verifica si existe retraso respecto de la fecha de devolución establecida.

En caso de detectarse un retraso, el sistema calcula y registra la multa correspondiente de acuerdo con las políticas definidas por la institución. Finalmente, se actualiza la disponibilidad del ejemplar dentro del catálogo y se envía una notificación al usuario confirmando la devolución realizada.

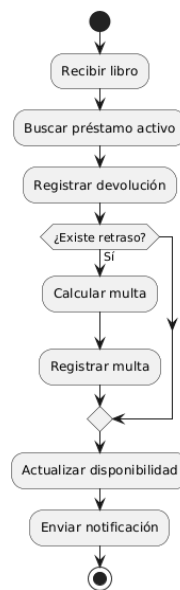


Figura 6. Diagrama de Actividad – Registro de Devolución.

Este diagrama permite modelar el comportamiento dinámico del sistema durante el proceso de devolución, asegurando la correcta gestión de préstamos, multas y disponibilidad de recursos bibliográficos.

4.9. Diagrama de Secuencia – Registro de Devolución

El diagrama de secuencia representa la interacción temporal entre los distintos actores y componentes del sistema durante el proceso de devolución de material bibliográfico. El flujo comienza cuando el bibliotecario registra la devolución del ejemplar mediante la interfaz de usuario, generando una solicitud que es enviada a la API REST para su procesamiento.

Posteriormente, el servicio de préstamos valida la existencia del préstamo activo y consulta la información almacenada en la base de datos. Si corresponde, el servicio de multas verifica la existencia de retrasos y registra las sanciones asociadas al usuario. Una vez completadas las validaciones, el sistema actualiza el estado del ejemplar como disponible dentro del catálogo y genera una notificación de confirmación.

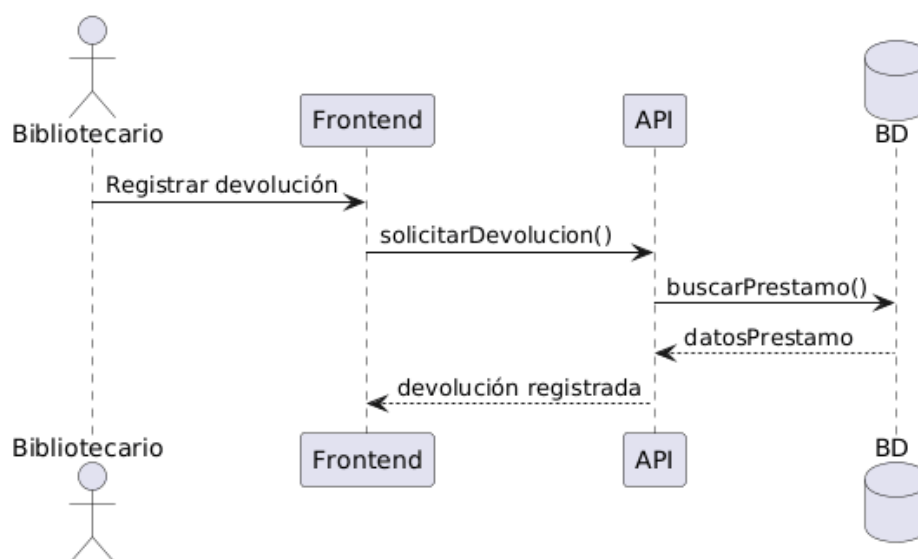


Figura 7. Diagrama de Secuencia – Registro de Devolución.

Este diagrama permite visualizar la comunicación entre los diferentes componentes de la arquitectura del sistema, facilitando la comprensión de las responsabilidades de cada módulo y asegurando la correcta ejecución de las reglas de negocio relacionadas con el proceso de devolución.

4.10. Diagrama de Despliegue

El diagrama de despliegue muestra la infraestructura propuesta para ejecutar UNILIB. Explica cómo se distribuyen el cliente web, el servidor de aplicación, la base de datos, el correo institucional y los respaldos.

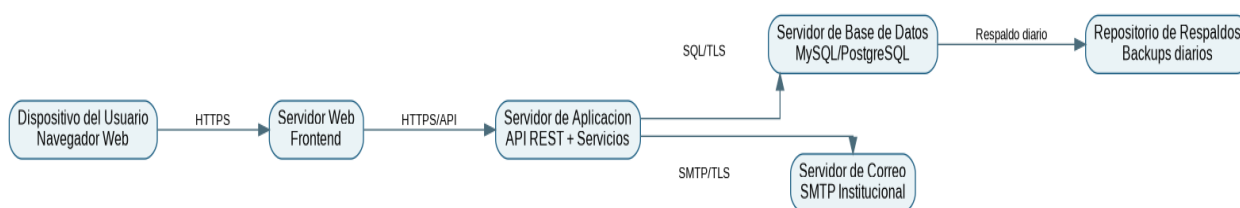


Figura 8. Diagrama de Despliegue de UNILIB.

El usuario accede desde un navegador mediante HTTPS. El servidor web entrega la interfaz, el servidor de aplicación ejecuta la API REST y los servicios, la base de datos almacena la información y el servidor SMTP envía correos. Los respaldos diarios apoyan la continuidad operacional.

4.11. Modelo Arquitectónico 4+1

El modelo 4+1 permite describir UNILIB desde múltiples perspectivas complementarias. Cada vista responde a una preocupación distinta del sistema y se valida con escenarios de uso.

Vista 4+1	Aplicacion en UNILIB	Artefactos relacionados
Vista lógica	Representa las funcionalidades del sistema y las entidades del dominio: usuarios, libros, ejemplares, prestamos, reservas, multas, reportes y notificaciones.	Diagrama de clases y casos de uso.
Vista de procesos	Describe flujos dinámicos, concurrencia y comportamiento operacional: solicitud de préstamo, devolución, cálculo de multas y envío de notificaciones.	Diagramas de actividad, diagramas de secuencia y reglas de negocio.
Vista de desarrollo	Organiza el código y componentes: frontend, API REST, servicios de negocio, persistencia y base de datos.	Diagrama de componentes, módulos y repositorio Git.
Vista fisica	Mapea el software a infraestructura: navegador, servidor web, servidor de aplicación, base de datos, correo y respaldos.	Diagrama de despliegue.
Vista de escenarios (+1)	Integra y valida las vistas anteriores mediante casos de uso reales: buscar libro, solicitar préstamos, registrar devolución y generar reporte.	Casos de uso, pruebas de aceptación y UAT.

La aplicación del modelo 4+1 contribuye a la integración del sistema porque conecta requisitos, componentes, infraestructura y escenarios de validación. De esta forma, el equipo puede verificar que la arquitectura propuesta satisface las necesidades del negocio y los requisitos técnicos.

La Vista de Procesos se encuentra representada mediante los diagramas de actividad y secuencia correspondientes a los procesos de solicitud de préstamo y registro de devolución. Estos diagramas permiten modelar tanto el flujo de actividades como la interacción entre los componentes que participan en la ejecución de las operaciones principales del sistema, contribuyendo a la comprensión del comportamiento dinámico de la solución propuesta.

4.12. Patrones de Diseño y Arquitectónicos Aplicados

Patron / Enfoque	Uso propuesto en UNILIB	Beneficio
Arquitectura cliente-servidor	El navegador consume servicios publicados por la API REST.	Permite acceso remoto y separación entre interfaz y lógica de negocio.
Arquitectura por capas	Separación entre presentación, aplicación/API, servicios de dominio y persistencia.	Mejora mantenibilidad, pruebas y escalabilidad.
MVC en la capa de interfaz	La interfaz separa vistas, manejo de eventos/controladores y modelos de datos recibidos desde la API.	Ordena el desarrollo frontend y reduce duplicidad.
Servicios y repositorios	La lógica de negocio queda en servicios; el acceso a datos queda en repositorios.	Aumenta encapsulamiento y facilita pruebas unitarias.
4+1	Se documentan vistas lógicas, procesos, desarrollo, física y escenarios.	Entrega una visión integral de arquitectura y facilita comunicación técnica.

4.13. Prototipo de Interfaces

Las interfaces de UNILIB fueron armadas como un prototipo funcional web, hecho con HTML, CSS y JavaScript, y publicado en GitHub Pages para que se pueda revisar y probar de forma fácil. En este prototipo se muestran las partes más importantes del sistema, como el inicio de sesión, el catálogo, las reservas, los préstamos y el historial, para representar de forma simple cómo funcionaría la solución en un escenario real. Aunque todavía no tiene backend ni una base de datos conectada, sí sirve para validar la navegación, la estructura visual y la relación entre los requisitos del sistema y la experiencia del usuario. También permite comprobar que el diseño se mantiene responsivo y alineado con los requisitos funcionales y no funcionales definidos en el informe.



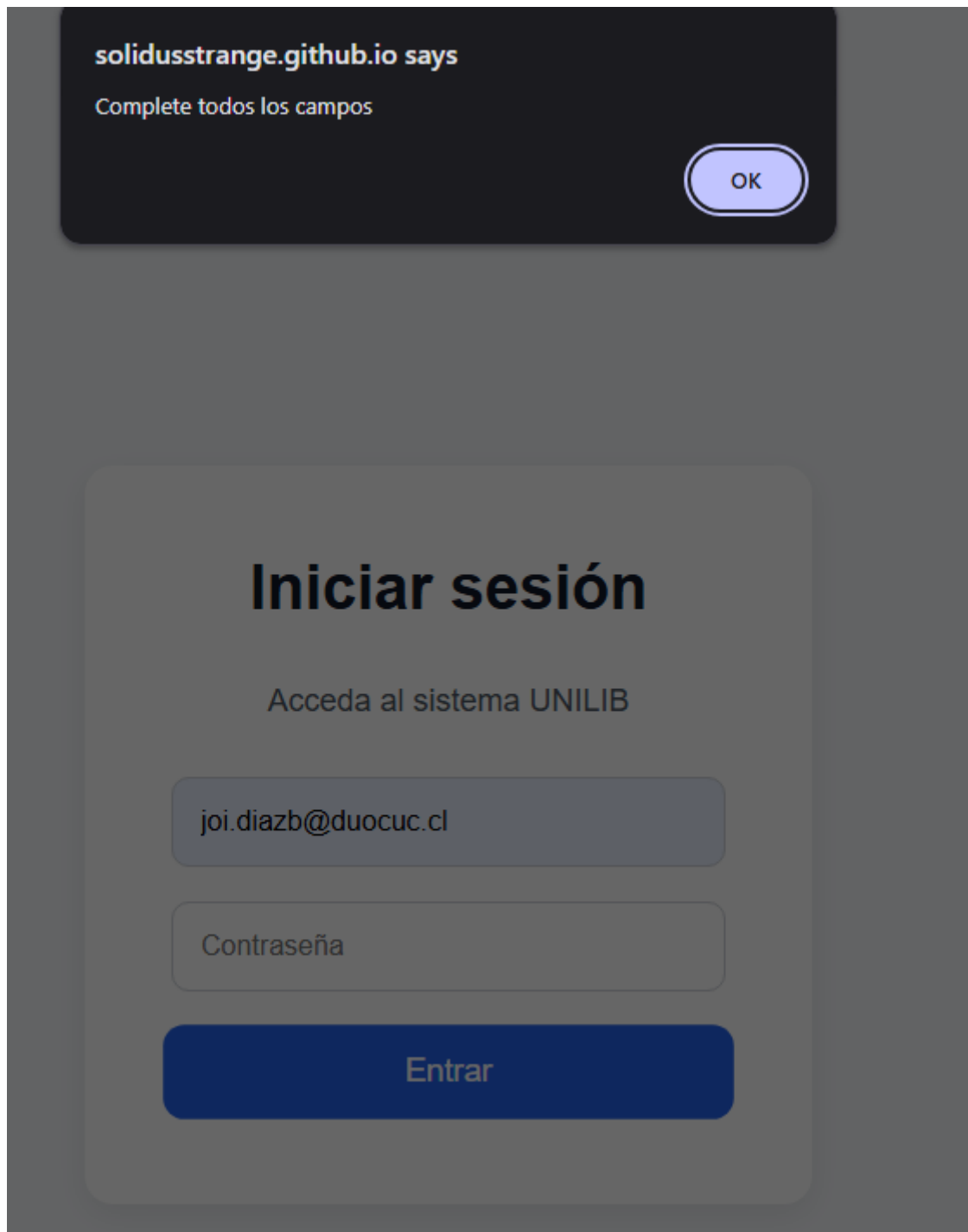
Pantalla de inicio – index.html

Iniciar sesión

Acceda al sistema UNILIB

Entrar

Acceso al sistema – login.html



The image shows a web browser window with a dark-themed login page. At the top, a dark notification bar displays the message "solidusstrange.github.io says" in yellow, followed by "Complete todos los campos" in white, and an "OK" button. The main content area has a light gray background with a white rounded rectangle containing the login form. The form title is "Iniciar sesión" in large black font, followed by the subtitle "Acceda al sistema UNILIB" in smaller gray font. There are two input fields: the first contains the email "joi.diazb@duocuc.cl" and the second is labeled "Contraseña". Below these is a blue "Entrar" button.

solidusstrange.github.io says
Complete todos los campos

Iniciar sesión

Acceda al sistema UNILIB

joi.diazb@duocuc.cl

Contraseña

Entrar

Prueba de mal ingreso de información – login.html

UNILIB Dashboard



523

Libros disponibles



91

Préstamos activos



120

Usuarios registrados

Consultar libros

Catalogo – dashboard.html

Catálogo de Libros

Buscar libro por título

Clean Code

Autor: Robert C. Martin

Estado: Disponible

Solicitar préstamo

1984

Autor: George Orwell

Estado: Prestado

Devolver

Clean Code

Autor: Robert C. Martin

Estado: Prestado

Devolver

1984

Autor: George Orwell

Estado: Prestado

Devolver

5. Propuesta de Planificación

5.1. Descripción General de la Planificación

El proyecto UNILIB se desarrollará en un periodo de 8 semanas calendario con dedicación parcial del equipo. La planificación se basa en el marco ágil Scrum, organizando el trabajo en iteraciones de dos semanas que permitan entrega incremental de valor y retroalimentación continua.

El proyecto comprende cinco fases principales: análisis de requisitos, diseño del sistema, desarrollo iterativo, pruebas y aseguramiento de calidad, y documentación final. Algunas fases se ejecutarán en paralelo para optimizar el calendario.

La metodología Scrum permite gestionar cambios en los requisitos sin comprometer la calidad del entregable, aplicando revisiones al final de cada sprint y ciclos de mejora continua.

5.2. Definición del Equipo de Trabajo

Integrante	Rol	Responsabilidades
_____	Jefe de Proyecto	Planificación, seguimiento de avances, gestión de riesgos y coordinación del equipo.
_____	Analista de Requisitos	Levantamiento y documentación de requisitos funcionales y no funcionales.
_____	Desarrollador Backend	Diseño de base de datos, API REST y lógica de negocio.
_____	Desarrollador Frontend	Implementación de interfaz de usuario y experiencia UX/UI.
_____	Tester / QA	Planificación y ejecución de pruebas funcionales, regresión y aceptación.

El equipo aplicara las siguientes buenas prácticas durante la ejecución del proyecto:

- Control de versiones mediante Git con flujo GitFlow.
- Reuniones de sincronización semanales de no más de 30 minutos.
- Revisión de código obligatoria antes de integrar cambios a la rama develop.
- Documentacion de decisiones técnicas relevantes en el repositorio del proyecto.

5.3. Definición de Actividades Principales del Proyecto

11. Análisis de requisitos (Semanas 1-2): levantamiento, documentación y validación de la ERS con stakeholders.
12. Diseño del sistema (Semanas 3-4): arquitectura de software, modelo de datos relacional, diseño de API REST, prototipado de interfaces y diagramas UML.
13. Desarrollo iterativo (Semanas 5-7): implementación de módulos priorizados: autenticación, catalogo, prestamos, notificaciones y reportes.
14. Pruebas y QA (Semana 8): pruebas funcionales, regresión, seguridad y aceptación de usuario.
15. Documentacion final (Semanas 7-8): entrega de ERS revisado, manual de usuario, código fuente documentado y evidencias de pruebas.

5.4. Tablero Kanban - Work Ítems del Proyecto

El equipo gestionara el backlog mediante un tablero Kanban con columnas: Backlog, En Progreso, En Revisión y Completado. Los principales Work Ítems son:

- WI-01: Como estudiante, quiero buscar libros por título o autor para encontrar lo que necesito.
- WI-02: Como estudiante, quiero solicitar un préstamo en línea para no tener que ir físicamente.
- WI-03: Como bibliotecario, quiero registrar devoluciones y aplicar multas automáticamente.
- WI-04: Como administrador, quiero generar reportes de uso mensual para tomar decisiones.
- WI-05: Como usuario, quiero recibir alertas antes del vencimiento de mi préstamo.
- WI-06: Como bibliotecario, quiero gestionar el catálogo desde el panel administrativo.
- WI-07: Como administrador, quiero crear y gestionar cuentas de usuarios con roles diferenciados.

5.5. Carta Gantt

La Carta Gantt presenta la programación estimada del proyecto en 8 semanas. La lógica aplicada para reducir los días lineales totales se basa en la identificación de dependencias reales entre actividades y el solapamiento controlado de trabajo.

Fase / Actividad	Sem 1	Sem 2	Sem 3	Sem 4	Sem 5	Sem 6	Sem 7	Sem 8
Análisis de requisitos	X	X						
Diseño del sistema			X	X				
Desarrollo iterativo					X	X	X	
Pruebas y QA								X
Documentacion final							X	X
Mantenimiento								X

La superposición de Pruebas y QA con Documentacion final en la semana 8 permite cerrar ambas líneas de trabajo simultáneamente, reduciendo el calendario total a 8 semanas de calendario efectivo.

5.6. CI/CD y Buenas Prácticas de Desarrollo Seguro

Para fortalecer la calidad del entregable, UNILIB considera prácticas de integración y despliegue continuo. Cada cambio de código debiera pasar por validaciones automáticas antes de integrarse a la rama principal.

- Pipeline CI: instalación de dependencias, ejecución de pruebas unitarias, validación de estilo y revisión de seguridad básica.
- Validación de datos en cliente y servidor para prevenir entradas invalidas o maliciosas.
- Manejo seguro de secretos mediante variables de entorno y exclusión de credenciales del repositorio.
- Registro de eventos relevantes sin exponer datos sensibles en errores.

- Principio de fallar seguro: ante excepciones, el sistema debe mostrar mensajes controlados y registrar detalles solo para el equipo técnico.

5.7. Estimación de Costos del Proyecto

Para asegurar la sustentabilidad financiera y la viabilidad del proyecto, se ha realizado una estimación base de costos considerando el equipo mínimo, la infraestructura de despliegue y las herramientas de apoyo requeridas para el prototipo y su validación.

Categoría de Costo	Descripción Técnica / Recurso	Esfuerzo / Cantidad	Costo Total (CLP)
Recursos Humanos	1 Arquitecto de Software / Tech Lead 2 Desarrolladores Full-Stack 1 QA	4 meses dedicados de desarrollo e integración	\$16.800.000
Infraestructura Cloud	Servicios de computación y base de datos relacional gestionada	12 meses de operación estimada	\$1.440.000
Continuidad y Seguridad	Almacenamiento remoto e inmutable para respaldos	12 meses de retención de datos	\$360.000
Herramientas y Licencias	Herramientas de análisis, testing y certificados SSL/TLS	Anual	\$600.000
COSTO TOTAL	Presupuesto general consolidado del proyecto UNILIB	Horizonte 1 año	\$21.120.000

5.8. Gestión de Configuración y Control de Versiones

- main: rama estable para entregas validadas.
- develop: integración diaria del equipo.
- feature/*: ramas temporales para requerimientos específicos.
- Pull Requests: integración obligatoria con revisión por pares antes de fusionar cambios.

La gobernanza del código fuente se realiza con Git y GitHub como repositorio central. El prototipo se despliega mediante GitHub Pages y cada cambio importante queda trazado con commits descriptivos, ramas de trabajo y revisiones previas a la integración.

5.9. Roles y Responsabilidades del Equipo

- Arquitectura y backend: diseño de la solución, lógica de negocio y API REST.
- Frontend y UX/UI: implementación de la interfaz y validación de usabilidad.
- QA: plan de pruebas, verificación de requisitos y evidencias.
- Coordinación: seguimiento de avances, control de versiones y entrega de hitos.

La distribución de responsabilidades se alinea con los roles definidos para la ejecución del proyecto y permite cubrir análisis, desarrollo, validación y documentación sin solapamientos críticos.

6. Estándar de Calidad Aplicado - ISO/IEC 25010

6.1. Selección del Estándar entre ISO, Nielsen y WCAG

Para el proyecto UNILIB se selecciona ISO/IEC 25010 como marco principal de calidad del software, debido a que permite evaluar el sistema de manera integral. Este estándar no se limita a la interfaz; también considera atributos como adecuación funcional, eficiencia de desempeño, confiabilidad, seguridad, usabilidad, mantenibilidad y portabilidad.

Las heurísticas de Jakob Nielsen y WCAG son relevantes, pero se usaran como criterios complementarios. Nielsen aporta criterios de usabilidad para mejorar la experiencia de usuario, mientras WCAG aporta criterios de accesibilidad web. Ambos apoyan dimensiones específicas de ISO/IEC 25010, pero no reemplazan el modelo principal de calidad.

Alternativa	Enfoque principal	Decision para UNILIB
ISO/IEC 25010	Calidad integral del producto software.	Seleccionado como estándar principal.
Nielsen	Usabilidad e interacción usuario-sistema.	Complementario dentro de Usabilidad.
WCAG	Accesibilidad web para personas con distintas capacidades.	Complementario dentro de Accesibilidad/Portabilidad.

6.2. Aplicación de ISO/IEC 25010 en UNILIB

Característica ISO/IEC 25010	Aplicación en UNILIB	Evidencia
Adecuación funcional	El sistema cubre las funciones necesarias para gestión bibliotecaria: usuarios, catalogo, prestamos, reservas, multas, reportes y notificaciones.	RF-01 a RF-14.
Eficiencia de desempeño	Las operaciones estándar deben responder en menos de 2 segundos bajo carga normal.	RNF-01.
Confiabilidad	El sistema debe mantener disponibilidad de 99.5% y recuperarse ante fallos en menos de 2 horas.	RNF-02 y respaldos diarios.
Seguridad	Uso de JWT, TLS, bcrypt, control de acceso por roles y auditoria de acciones críticas.	RNF-03, RF-13 y otros requisitos de auditoría.
Usabilidad	Interfaz intuitiva, navegación consistente, retroalimentación visual y curva de aprendizaje menor a 1 hora.	RNF-04 e interfaces de usuario.
Mantenibilidad	Arquitectura modular, código documentado, pruebas automatizadas y cobertura $\geq 80\%$.	RNF-06 y principios de modularidad.
Portabilidad	Sistema web responsive compatible con navegadores modernos y múltiples dispositivos.	RNF-05.

Escalabilidad	Soporte de crecimiento de 50% en usuarios sin degradación de rendimiento.	RNF-07.
---------------	---	---------

6.3. Complemento de Usabilidad: Heurísticas de Nielsen

Las heurísticas de Nielsen se aplican como apoyo para validar que la interfaz sea clara, predecible y eficiente para los distintos tipos de usuario.

Heurística aplicada	Criterio en UNILIB
Visibilidad del estado del sistema	Confirmaciones, mensajes de error y estados de solicitud en menos de 2 segundos.
Coincidencia con el mundo real	Uso de conceptos familiares para usuarios de biblioteca: catalogo, préstamo, reserva, multa e historial.
Prevención de errores	Validaciones de formularios, campos obligatorios y confirmación antes de acciones críticas.
Consistencia y estándares	Menú principal constante y terminología uniforme en todos los módulos.
Ayuda al usuario a reconocer y corregir errores	Mensajes claros que indiquen causa y acción sugerida.

6.4. Complemento de Accesibilidad: WCAG 2.1 Nivel AA

WCAG se integra como complemento para asegurar que el sistema pueda ser utilizado por la mayor cantidad posible de usuarios. En UNILIB se consideran los principios perceptible, operable, comprensible y robusto.

Principio WCAG	Aplicación en UNILIB
Perceptible	Contraste mínimo 4.5:1, texto legible y estructura visual clara.
Operable	Navegación consistente, formularios claros y compatibilidad con teclado.
Comprensible	Mensajes simples, etiquetas de campos y flujos predecibles.
Robusto	Compatibilidad con navegadores modernos y estructura HTML semántica para futuras ayudas técnicas.

6.5. Matriz de Verificación de Calidad

Requisito	Criterio de calidad	Forma de verificación
RNF-01	Eficiencia de desempeño	Pruebas de carga y medición de tiempo de respuesta.
RNF-02	Confiabilidad	Monitoreo de disponibilidad y simulación de recuperación.
RNF-03	Seguridad	Revisión de autenticación, cifrado, roles y auditoria.
RNF-04	Usabilidad	Pruebas con usuarios y checklist Nielsen.
RNF-05	Portabilidad/Accesibilidad	Pruebas responsive y checklist WCAG nivel AA.
RNF-06	Mantenibilidad	Revisión de cobertura de pruebas, estructura modular y documentación.
RNF-07	Escalabilidad	Pruebas con aumento simulado de usuarios y medición de degradación.

La aplicación de ISO/IEC 25010 contribuye a la calidad del software porque transforma los requisitos no funcionales en criterios medibles, verificables y trazables. Esto permite que el

equipo no solo declare calidad, sino que pueda demostrarla mediante pruebas, revisiones y evidencia técnica.

6.6. Plan de Pruebas del Prototipo

El plan de pruebas considera casos funcionales y no funcionales sobre el prototipo web. Cada caso se asocia a un requisito, registra el resultado esperado y exige evidencia verificable de la ejecución. Se priorizan autenticación, consulta del catálogo, préstamo, accesibilidad y control de versiones.

7. Anexos

7.1. Matriz de Especificación de Requisitos

La siguiente matriz consolida todos los requisitos funcionales y no funcionales con sus identificadores, descripción resumida, prioridad, estado, tipo y fuente de origen.

ID	Nombre	Descripcion	Prioridad	Estado	Tipo	Fuente
RF-01	Registro de usuarios	Alta, baja y modificación con validación y roles.	Alta	Propuesto	RF	Entrevista
RF-02	Autenticación	Login seguro con credenciales únicas y cifrado.	Alta	Propuesto	RF	Entrevista
RF-03	Busqueda catalogo	Filtros múltiples y disponibilidad en tiempo real.	Alta	Propuesto	RF	Taller req.
RF-04	Solicitud prestamos	Flujo solicitud, aprobación y comprobante.	Alta	Propuesto	RF	Taller req.
RF-05	Registro devoluciones	Entrega, fechas y calculo de multas automático.	Alta	Propuesto	RF	Taller req.
RF-06	Renovación prestamos	Renovación online si no hay reservas pendientes.	Media	Propuesto	RF	Entrevista
RF-07	Reservas	Reserva de material no disponible con notificación.	Media	Propuesto	RF	Entrevista
RF-08	Notificaciones	Alertas automáticas por vencimiento y disponibilidad.	Alta	Propuesto	RF	Taller req.
RF-09	Gestión catalogo	CRUD de registros bibliográficos y ejemplares.	Alta	Propuesto	RF	Entrevista
RF-10	Reportes	Exportación PDF/Excel de estadísticas del sistema.	Media	Propuesto	RF	Entrevista

RF-11	Gestión de multas	Calculo, registro y consulta de multas por retrasos.	Alta	Propuesto	RF	Entrevista
RF-12	Historial de usuario	Consulta del historial de préstamos, devoluciones y multas.	Media	Propuesto	RF	Entrevista
RF-13	Auditoria del sistema	Registro de acciones críticas con usuario y timestamp.	Alta	Propuesto	RF	Restricción
RF-14	Validación de datos	Validación de campos obligatorios, formatos y duplicados.	Alta	Propuesto	RF	Estándar
RNF-01	Rendimiento	Respuesta < 2 seg. bajo carga normal.	Alta	Propuesto	RNF	Restricción
RNF-02	Disponibilidad	Uptime >= 99.5%; recuperación < 2 horas.	Alta	Propuesto	RNF	Restricción
RNF-03	Seguridad	JWT, TLS, bcrypt, control de acceso por roles.	Alta	Propuesto	RNF	Restricción
RNF-04	Usabilidad	Curva de aprendizaje < 1 hora para usuario básico.	Media	Propuesto	RNF	Estándar
RNF-05	Accesibilidad	Responsive, compatible con navegadores modernos.	Media	Propuesto	RNF	Estándar
RNF-06	Mantenibilidad	Cobertura de pruebas >= 80%, arquitectura modular.	Media	Propuesto	RNF	Estándar
RNF-07	Escalabilidad	Soporta 50% mas usuarios sin degradar rendimiento.	Baja	Propuesto	RNF	Restricción

7.2. Matriz de Cumplimiento de Rubrica 2

Indicador	Evidencia incorporada en el informe	Seccion
IE2.1.1	Aplicación de técnicas de modelado UML para representar arquitectura y diseño.	Secciones 4.1 a 4.7
IE2.2.1	Uso de patrones de diseño y modelo 4+1 para modelar requisitos y proponer solución arquitectónica adaptativa.	Secciones 4.8 y 4.9
IE2.3.1	Definición de estándares de calidad en el diseño de software con ISO/IEC 25010.	Sección 6
IE2.1.2	Exposición de técnicas de modelado UML utilizadas en el proyecto.	Secciones 4.1 a 4.7
IE2.2.2	Explicación del modelo 4+1 y su aporte al desarrollo e integración del sistema.	Sección 4.8
IE2.2.3	Explicación de diagramas de Caso de Uso, Clase, Componentes, Actividad y Despliegue.	Secciones 4.3 a 4.7
IE2.3.2	Argumentación de estándares de calidad aplicados y contribución a la calidad del software.	Secciones 6.1 a 6.5

7.3. Matriz de Cumplimiento de Rubrica 3

Punto de la rúbrica	Evidencia en UNILIB	Ubicación	Estado
14	Prototipo funcional web con JavaScript y despliegue en GitHub Pages	4.13	Cumple
15	Diseño del prototipo considerando RF y RNF	4.13 / 6.5 / 7.4	Cumple
16	Plan de pruebas según normativa	6.6 / 7.4	Cumple
17	Planilla de casos de prueba y evidencias	7.4	Cumple
18	Registro de control de cambios y versiones	5.8 / 7.5	Cumple

7.4. Planilla de Casos de Prueba y Evidencias

ID	Tipo	Caso de prueba	Resultado esperado	Evidencia
TC-01	Funcional	Validar inicio de sesión con credenciales válidas	Acceso correcto al sistema y carga del menú principal	Captura de pantalla / registro de sesión
TC-02	Funcional	Buscar materiales por título o autor	Listado de resultados coincidente con el filtro aplicado	Captura de pantalla del catálogo
TC-03	Funcional	Solicitar préstamo desde el catálogo	Solicitud registrada con estado pendiente o aprobada	Pantalla de confirmación
TC-04	No funcional	Verificar diseño responsive en móvil y escritorio	La interfaz se adapta sin pérdida de contenido	Capturas en dos resoluciones
TC-05	No funcional	Comprobar navegación básica del prototipo publicado	Los módulos principales responden sin errores de carga	Enlace a GitHub Pages

7.5. Registro de Control de Cambios y Versiones

Versión	Rama / Tag	Cambio realizado	Motivo	Estado
v0.1	feature/prototipo	Estructura inicial del prototipo web	Validar navegación básica	En revisión
v0.2	develop	Se agregan catálogo y préstamos	Cubrir RF principales	En revisión
v1.0	main / tag v1.0.0	Versión consolidada para entrega	Cerrar evidencia final	Aprobada

8. Propuestas de Mejora y Aseguramiento de Calidad

Con el objetivo de garantizar la mejora continua del sistema UNILIB y aumentar su valor para la institución, se proponen las siguientes acciones:

8.1. Mejoras Funcionales

- Sistema de recomendaciones inteligentes: modulo que sugiera libros en base al historial de préstamos del usuario. Beneficio esperado: mejora la experiencia del usuario y aumenta el uso del sistema.
- Aplicación móvil: desarrollo de una versión para iOS y Android. Beneficio esperado: acceso más rápido y mayor adopción por parte de los usuarios.
- Dashboard analítico avanzado: visualización de datos mediante gráficos interactivos. Beneficio esperado: facilita la toma de decisiones estratégicas en la biblioteca.

8.2. Mejoras Técnicas

- Optimización de base de datos: uso de índices y consultas optimizadas para mejorar rendimiento.
- Implementación de cache: almacenamiento temporal de consultas frecuentes para disminuir carga del servidor.
- Escalabilidad en la nube: despliegue en infraestructura cloud para permitir crecimiento sin pérdida de rendimiento.
- Automatización CI/CD: ejecución automática de pruebas y validaciones antes de liberar cambios.

8.3. Aseguramiento de Calidad

- Pruebas automatizadas: implementación de pruebas unitarias, integradas y de regresión.
- Monitoreo del sistema: herramientas de monitoreo en tiempo real para detección temprana de fallos.
- Auditoria de seguridad periódica: evaluación continua de vulnerabilidades y revisión de logs.
- Pruebas de accesibilidad y usabilidad: revisión con checklist WCAG y heurísticas de Nielsen.
- Revisión arquitectónica: validación de que los componentes mantienen alta cohesión, bajo acoplamiento y trazabilidad con los casos de uso.

8.4. Conclusiones del Modelo de Calidad

En UNILIB, esta base de calidad favorece una solución más estable, mantenible y alineada con las necesidades de la comunidad universitaria. El prototipo, las pruebas y el control de versiones quedan así integrados como evidencia técnica del cumplimiento de la rúbrica 3.

La adopción integrada de ISO/IEC 25010, complementada con las heurísticas de Nielsen y WCAG, transforma criterios cualitativos en indicadores verificables. Esto reduce la ambigüedad del diseño y permite evaluar rendimiento, confiabilidad, seguridad, usabilidad y mantenibilidad con métricas concretas antes de pasar a la construcción completa.

9. Conclusión

Con este informe se logró dejar ordenado todo lo necesario para UNILIB: requisitos, modelado, planificación, calidad y ahora también el prototipo, el plan de pruebas y el control de versiones. En conjunto, esto permite ver mejor cómo se relaciona cada parte del proyecto y cómo la solución responde a lo que necesita una biblioteca universitaria. El prototipo cumple un rol importante porque muestra de forma práctica la idea del sistema y ayuda a comprobar si la propuesta funciona bien antes de pasar a una implementación completa. En resumen, el trabajo deja una base bastante clara para seguir avanzando, con una propuesta coherente, verificable y alineada con la rúbrica 3.